

Programski jezici

-Pripremna laboratorijska vežba-

-C#-

NAPOMENA:

Svaki od zadataka treba rešiti korišćenjem ili interfejsa ili apstraktne klase, u zavisnosti od sadržine zadatka. Svaka od klasa koja se kreira treba da sadrži konstruktor u kome će se postavljati vrednosti svih atributa. Sve zadatke implementirati na programskom jeziku C#.

ZADACI:

Grupa 1

Projektovati sistem za potrebe HR odseka jedne kompanije. U klasi Kompanija treba da se čuva lista zaposlenih koju treba implementirati kreiranjem generičke klase Kolekcija koja će da pamti listu (List<T>) elemenata tipa Zaposleni ili tipa izvedenog iz klase Zaposleni. Svakog zaposlenog karakteriše ime, prezime i plata. Na osnovu pozicije na kojoj rade zaposleni razlikuju se WebDeveloperi, QA inženjeri i DevOps inženjeri. Pored svih informacija koje ima Zaposleni, WebDeveloper dodatno treba da čuva listu tehnologija (tekstualni podaci) koje poznaje, QA inženjer treba da ima postavljen broj QA testova koje može da uradi na dnevnom nivou, a DevOps inženjer treba da pamti datum kada je poslednji put pregledao računare. Za svaku od ovih pozicija treba omogućiti da se prikazuju informacije ove pozicije, odnosno da se štampa ona informacija koja je njemu svojstvena. Svakom od zaposlenih se u trenutku kada je to potrebno obračunava bonus i to za WebDevelopere 20% visine plate ako zna više od 5 tehnologija inače 10%, za QA 10% ako radi više od 10 testova na dnevnom nivou inače 5%, a za DevOps 30% ako je pregledao računare u proteklih 7 dana inače 15%. Omogućiti čitanje i upis svih informacija o Kompaniji korišćenjem klase BinaryReader, odnosno BinaryWriter. U klasi Kompanija definisati konstantu pod imenom maxBonus i omogućiti da se, ukoliko se proračunati bonus prekorači, baci izuzetak PrevelikiBonus.

Dodatno treba preklopiti operator + koji dodaje novog zaposlenog u listu. U svim klasama iskoristiti svojstva (property) za pristup atributima klase.

U Main metodi kreirati instancu klase Kompanija sa određenim brojem zaposlenih. Dalje treba testirati metode za rad sa fajlovima u Main metodi, tako što se prvo upiše instanca klase Kompanija u fajl, a nakon toga se pročita sadržaj fajla i smesti u novi objekat tipa Kompanija. Kada je sadržaj fajla pročitao, štampati sve podatke o Kompaniji na standardni izlaz da biste demonstrirali da sve metode rade (prikazati sve podatke o zaposlenima i odrediti bonuseve za zaposlene). Čitanje iz fajla i upisivanje u fajl ne sme biti obavljeno u Main-u, već u odgovarajućim funkcijama koje treba pozvati u Main-u. Prilikom rada sa fajlovima voditi računa o zatvaranju tokova.

Grupa 2

Projektovati sistem za pomoć pri parkiranju vozila na javnom parkingu. Svako vozilo karakteriše registarska oznaka, proizvođač i naziv modela. Vozilo samo određuje koliko standardnih parking mesta mu je potrebno. Automobil zauzima uvek jedno parking mesto, autobus zauzima uvek 5 parking mesta, a kamion može da zauzme različit broj parking mesta (čuva taj broj kao atribut klase). Dodatni atribut autobusa je broj putnika, a dodatni atribut kamiona nosivost u tonama. Parking čuva parking mesta kao generičku klasu `Matrica<T>` koja će u sebi pamti matricu elemenata tipa `Vozilo` ili tipa izvedenog iz klase `Vozilo` sa N vrsta i M kolona. Za vozila koja zauzimaju više od jednog parking mesta ta mesta treba da se nalaze u istoj vrsti i jedno pored drugog. Vozilo ulazi na parking tako što poziva odgovarajuću metodu klase `Parking` kojoj prosleđuje broj mesta koji mu je potreban. Ako se pronađe odgovarajući broj praznih mesta reference sa tih mesta se postavljaju tako da pokazuju na to uparkirano vozilo. Ako `Parking` ne pronađe odgovarajući broj praznih mesta omogućiti bacanje izuzetka tipa `NemaMesta`. Vozilo izlazi sa parkinga tako što poziva odgovarajuću metodu klase `Parking` u kojoj `Parking` pronalazi to vozilo i prazni sva mesta koja je to vozilo zauzimalo. `Parking` ima i mogućnost da prikaže trenutno stanje svojih parking mesta (da li su zauzeta ili slobodna). Omogućiti čitanje i upis svih informacija o Parkingu korišćenjem klase `BinaryReader`, odnosno `BinaryWriter`.

Definisati nad klasom `Parking` indeksere koji će da vraća vozilo sa zadatim indeksima iz matrice vozila (`parking[i,j]`). U svim klasama iskoristiti svojstva (`property`) za pristup atributima klase.

U `Main` metodi kreirati instancu klase `Parking` sa određenim brojem vrsta i kolona i nekoliko vozila, uparkirati ta vozila na `Parking`, prikazati stanje na parkingu, a zatim ih isparkirati sa `Parkinga`. U klasi `Parking` jednostavnosti radi ne voditi računa o ukupnjavanju praznih mesta, već se vozila smeštaju na prvo parking mesto koje zadovoljava njihove zahteve. Dalje treba testirati metode za rad sa fajlovima u `Main` metodi, tako što se prvo upiše instanca klase `Parking` u fajl, a nakon toga se pročita sadržaj fajla i smesti u novi objekat tipa `Parking`. Kada je sadržaj fajla pročitan štampati sve podatke o `Parkingu` na standardni izlaz da biste demonstrirali da sve metode rade. Čitanje iz fajla i upisivanje u fajl ne sme biti obavljeno u `Main-u`, već u odgovarajućim funkcijama koje treba pozvati u `Main-u`. Prilikom rada sa fajlovima voditi računa o zatvaranju tokova.

Grupa 3

Projektovati sistem za sređivanje spiska literature na kraju neke knjige ili članka u časopisu. Klasa Literatura treba da sadrži listu stavki iz literature koju treba implementirati kreiranjem generičke klase Kolekcija koja će da pamti listu (List<T>) elemenata tipa Stavka ili tipa izvedenog iz klase Stavka. U klasi Literatura definisati konstantu maxStavke. Baciti NemaMesta izuzetak ukoliko se proba dodavanje nove Stavke u Literaturu, a Literatura je već popunjena. Jedna stavka treba da vraća svoju godinu objavljivanja i da vraća tekst koji sadrži sve bitne podatke o njoj. Stavka u literaturi može da predstavlja knjigu i za nju se čuva naziv, lista autora, mesto i godina izdanja. Stavka u literaturi može da predstavlja članak iz časopisa i za njega se čuva naziv časopisa, naziv članka, lista autora, mesto i godina izdanja. Stavka u literaturi može da bude i web stranica i za nju se čuva samo adresa (link) i godina objavljivanja. Klasa Literatura treba da omogući sortiranje stavki po godini izdanja u rastućem ili opadajućem redosledu, sortiranje stavki po njihovoj tekstualnoj reprezentaciji u rastućem ili opadajućem redosledu (po abecedi) i prikaz svih stavki na standardnom izlazu. Omogućiti čitanje i upis svih informacija o Literaturi korišćenjem klase StreamReader, odnosno StreamWriter.

Definisati nad klasom Literatura indekserski koji će da vraća stavku sa zadatim indeksom iz kolekcije stavki. U svim klasama iskoristiti svojstva (property) za pristup atributima klase.

U Main metodi kreirati instancu klase Literatura sa određenim brojem stavki, sortirati stavke po tekstu u rastućem redosledu i prikazati sve stavke iz literature. Dalje treba testirati metode za rad sa fajlovima u Main metodi, tako što se prvo upiše instanca klase Literatura u fajl, a nakon toga se pročita sadržaj fajla i smesti u novi objekat klase Literatura. Kada je sadržaj fajla pročitao štampati sve podatke o Literaturi na standardni izlaz da biste demonstrirali da sve metode rade. Čitanje iz fajla i upisivanje u fajl ne sme biti obavljeno u Main-u, već u odgovarajućim funkcijama koje treba pozvati u Main-u. Prilikom rada sa fajlovima voditi računa o zatvaranju tokova.